LLMs

Complete Developer Study Notes

Concepts • Practical Examples • Projects • Interview Preparation

Prepared by: Rahul Yadav

20 Pages

Rahul Yadav

1. Introduction to LLMs

1.1 What are LLMs ?

- LLMs (Large Language Models) are deep learning models trained on massive amounts of text data to understand, generate and work with human language.
- They learn patterns, grammar, facts, reasoning and general knowledge from data using Transformer architecture.
- LLMs can perform various tasks such as text generation, summarization, translation, Q&A, code generation, reasoning and more.

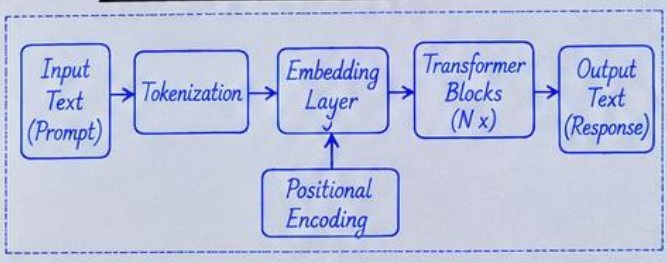
1.2 Why LLMs are Important ?

- Traditional ML models are task-specific and require manual feature engineering.
- LLMs are general-purpose models that can handle multiple tasks with the same model.
- They enable natural human-computer interaction.
- They can understand context, follow instructions and generate useful, coherent responses.
- LLMs power modern AI applications like chatbots, copilots, agents and automation tools.

1.3 Key Capabilities of LLMs

- Understand and generate human-like text.
- Follow natural language instructions.
- Reason over complex information.
- Perform zero-shot and few-shot learning.
- Work with multiple domains (general knowledge, code, math, etc.).
- Can be extended with tools, APIs and external data.
- Support for multi-turn conversations and memory (using advanced techniques).

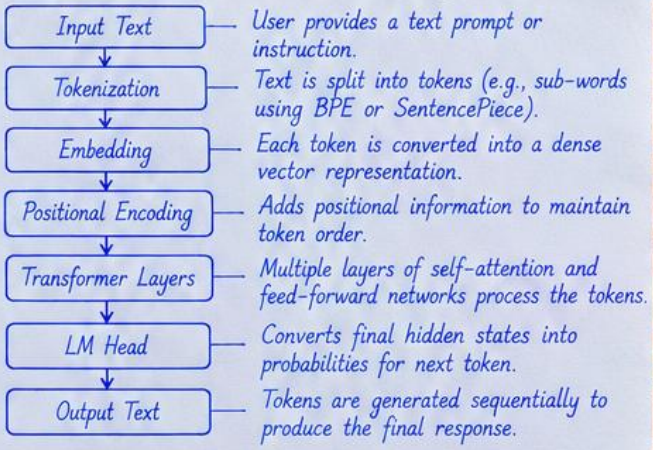
1.4 Basic Architecture



1.5 Core Components

- **Tokenizer** : Converts text into tokens (sub-words).
- **Embeddings** : Convert tokens into dense vectors.
- **Positional Encoding** : Adds position information to tokens.
- **Transformer Blocks** : Consists of self-attention, feed-forward layers, residual connections and layer normalization.
- **LM Head** : Converts final hidden states to probabilities of next token.
- **Output** : Generates text token by token using decoding strategies (e.g., Greedy, beam search, sampling).

1.6 How LLMs Work ? (Step by Step)



1.7 Types of LLMs

- **Base Models** : Pre-trained on large text data (e.g., GPT, LLaMA).
- **Instruction-Tuned Models** : Further trained to follow instructions.
- **Chat Models** : Optimized for conversational interactions.
- **Code Models** : Trained on code data (e.g., Code LLaMA, StarCoder).
- **Multimodal Models** : Can work with text, images, audio, etc.

1.8 Popular LLMs

Model	Organization	Key Features
GPT-3.5 / GPT-4	OpenAI	General purpose, chat, reasoning
LLaMA	Meta	Open source, research and fine-tuning
Gemini	Google	Multimodal, strong reasoning
Claude	Anthropic	Safe and helpful responses
Mistral	Mistral AI	Efficient and high performance
Falcon	TII	Open source, good performance

1.9 Real-world Applications

- Chatbots and virtual assistants.
- Content generation (blogs, emails, reports).
- Summarization (documents, articles, meetings).
- Code generation and software development.
- Q&A systems and knowledge assistants.
- Data analysis and report generation.
- Workfow automation using agents and tools.

1.10 Challenges and Limitations

- Can generate incorrect or hallucinated information.
- Limited by training data cutoff.
- May produce biased or harmful content.
- Struggles with very recent or domain-specific information (without RAG or fine-tuning).
- High computational and memory requirements.
- Requires careful prompt design and evaluation.

1.11 Summary

- LLMs are powerful general-purpose models for understanding and generating human language.
- They are based on Transformer architecture and trained on large text datasets.
- They can be applied to a wide range of real-world use cases but also have certain limitations.

Rahul Yadav

2. Tokenization in LLMs

2.1 What is Tokenization ?

- Tokenization is the process of converting raw text into smaller units called tokens.
- Tokens can be words, sub-words or even characters depending on the tokenizer used.
- LLMs do not understand text directly, they work with token IDs (numbers) instead of text.
- Tokenization helps in handling unknown words, reducing vocabulary size and making the model more efficient.

2.2 Why Tokenization is Important ?

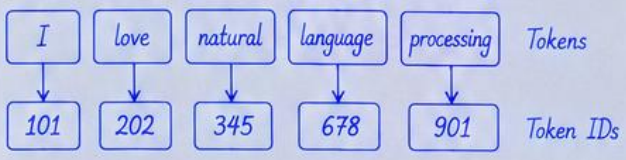
- Converts human language to numeric representation for model input.
- Helps handle new or rare words using sub-word techniques.
- Maintains the meaning of text while reducing vocabulary size.
- Enables the model to process multiple languages.
- Affects the model's performance, speed and context understanding.

2.3 Types of Tokenization

- **Word-based** : Splits text into whole words.
- **Character-based** : Splits text into individual characters.
- **Sub-word based** : Splits words into smaller meaningful pieces (most common in LLMs).

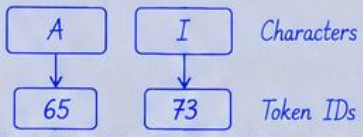
2.4 Word-based Tokenization (Example)

Text: "I love natural language processing"



2.5 Character-based Tokenization (Example)

Text: "AI"



- Simple and easy to implement.
- Vocabulary size is small (only characters).
- Results in longer sequences for the same text.

2.6 Sub-word Tokenization (Most Common)

- Used in modern LLMs like BERT, GPT, LLaMA, etc.
- Uses algorithms like BPE, WordPiece or Unigram LM.
- Splits rare words into meaningful sub-words.
- Helps in handling unknown words and multiple languages.

2.6.1 Example (Byte Pair Encoding - BPE)

Text: "unbelievable"

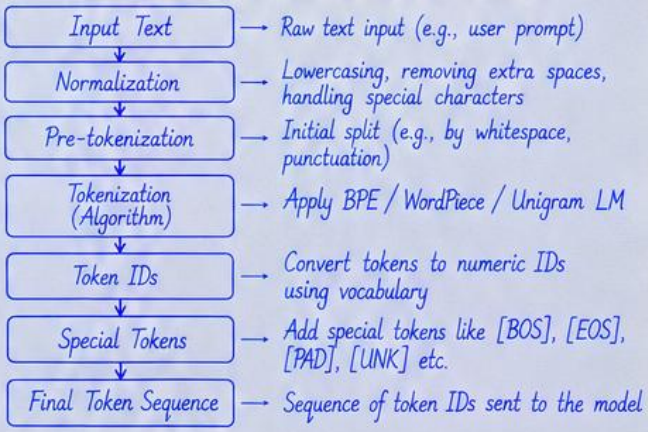


- Frequently occurring pairs of characters are merged to form sub-words.
- Reduces vocabulary size while keeping meaningful tokens.

2.7 Popular Tokenization Algorithms

Algorithm	Used In	Key Idea	Pros	Cons
BPE	GPT, LLaMA	Merge frequent byte pairs	Simple, effective	May create unusual tokens
WordPiece	BERT	Maximize likelihood	Good for masked LM	Slightly complex
Unigram LM	T5, SentencePiece	Probabilistic sub-words	Flexible, language agnostic	Needs more training data

2.8 Tokenization Process (Detailed Flow)



2.9 Special Tokens

Token	Purpose	Example
[BOS]	Beginning of sequence	<s> or [BOS]
[EOS]	End of sequence	</s> or [EOS]
[PAD]	Padding to fixed length	[PAD]
[UNK]	Unknown token	[UNK]
[SEP]	Separator between sentences	[SEP]
[CLS]	Classification token (mainly BERT)	[CLS]

2.10 Impact of Tokenization

- Affects model input length and memory usage.
- Influences model performance and generation quality.
- Different tokenizers produce different token counts.
- Important to use the same tokenizer as the pre-trained model.

Rahul Yadav

3. Embeddings in LLMs

3.1 What are Embeddings ?

- Embeddings are dense vector representations of tokens (words, sub-words or characters).
- They convert discrete text into continuous numerical vectors that capture semantic meaning.
- Similar words have similar vectors in the embedding space.
- Embeddings help LLMs understand the meaning, context and relationships between tokens.

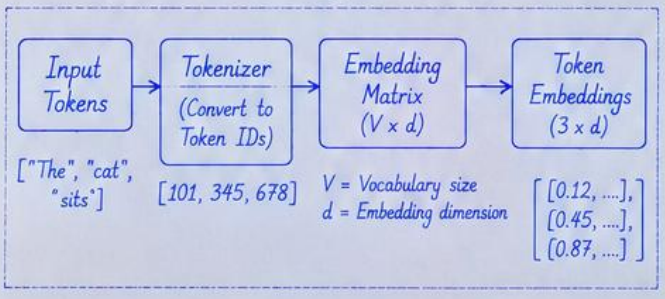
3.2 Why Embeddings are Important ?

- Machines cannot understand raw text directly.
- Embeddings convert text into numerical form that LLMs can process.
- They capture semantic similarity, context and syntactic information.
- Good embeddings improve model performance in tasks like reasoning, retrieval and generation.

3.3 Types of Embeddings

- **Token Embeddings** : Convert tokens into vectors using a lookup table or learned matrix.
- **Position Embeddings** : Capture the position of tokens in a sequence.
- **Segment Embeddings** : Used in models like BERT to distinguish different segments (e.g., sentence A and sentence B).
- **Combined Embeddings** : Sum of token + position (+ segment) embeddings.

3.4 Token Embedding (Architecture)



3.5 Embedding Matrix

- A learnable matrix of shape (V, d), where
  - V : Vocabulary size (e.g., 50,000)
  - d : Embedding dimension (e.g., 768, 1024)
- Each token ID maps to a unique d-dimensional vector.
- The matrix is learned during pre-training.
- Similar tokens get similar vectors based on training data.

3.6 Positional Embeddings

- Transformers do not have recurrence, so they need positional information to understand token order.
- Positional embeddings are added to token embeddings.
- Two common types:
  - Sinusoidal Positional Embeddings (used in original Transformer)
  - Learned Positional Embeddings (used in GPT, BERT, etc.)

3.6.1 Sinusoidal Positional Encoding (Formula)

$$PE(pos, 2i) = \sin\left(\frac{pos}{10000^{2i/d_{model}}}\right)$$

$$PE(pos, 2i+1) = \cos\left(\frac{pos}{10000^{2i/d_{model}}}\right)$$

pos : token position
i : dimension index
d_{model} : embedding dimension

3.6.2 Example of Positional Embeddings

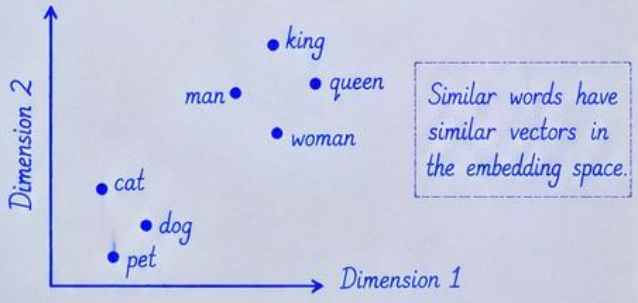
Position	Dimension 0	Dimension 1	Dimension 2	Dimension 3
0	0.0000	1.0000	0.0000	1.0000
1	0.8415	0.5403	0.0099	0.9999
2	0.9093	-0.4161	0.0198	0.9998
3	0.1411	-0.9900	0.0297	0.9996

3.7 Combined Embeddings

- The final input embedding is the sum of token embedding, positional embedding and (optional) segment embedding.

Input Embedding = Token Embedding
+ Positional Embedding
+ Segment Embedding (optional)

3.8 Visualization of Embedding Space



3.9 Key Takeaways

- Embeddings convert text into dense vectors.
- They capture semantic meaning and token relationships.
- Positional embeddings help maintain token order.
- The sum of embeddings is used as input to the Transformer.
- Good embeddings are crucial for LLM performance.

Rahul Yadav

4. Transformer Architecture Overview

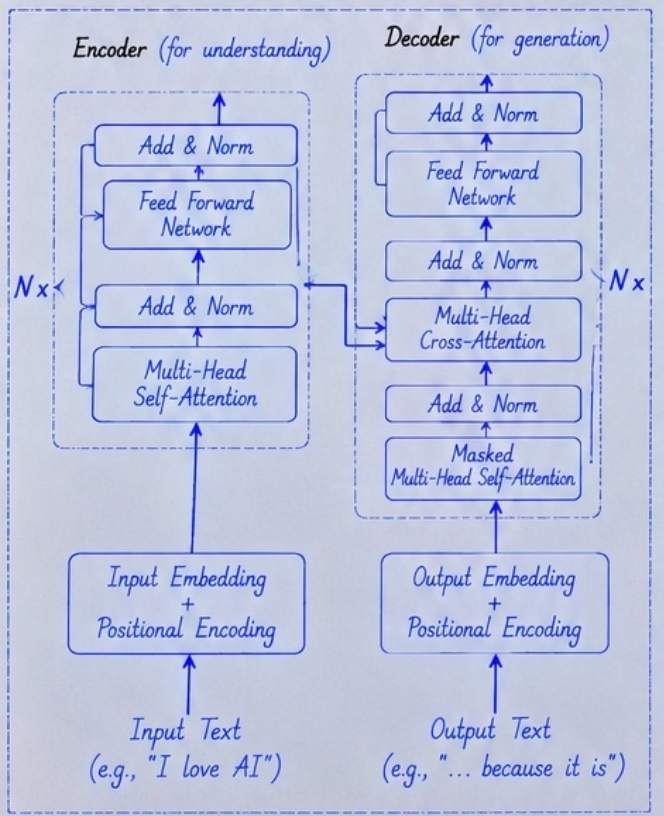
4.1 What is Transformer ?

- Transformer is a deep learning architecture introduced in the paper "Attention Is All You Need" (2017) by Vaswani et al.
- It is the foundation of modern LLMs like BERT, GPT, LLaMA, Gemini, Claude, etc.
- It uses self-attention mechanism instead of recurrent or convolutional layers.
- It can process input text in parallel, making it more efficient and scalable.
- It consists of multiple layers of self-attention and feed-forward networks with residual connections and layer normalization.

4.2 Key Characteristics

- No recurrence (unlike RNN/LSTM).
- Uses self-attention to capture long-range dependencies.
- Fully parallelizable (faster training).
- Works with variable length sequences.
- Highly scalable and effective for large datasets.
- Forms the base architecture for most modern LLMs.

4.3 High-Level Architecture



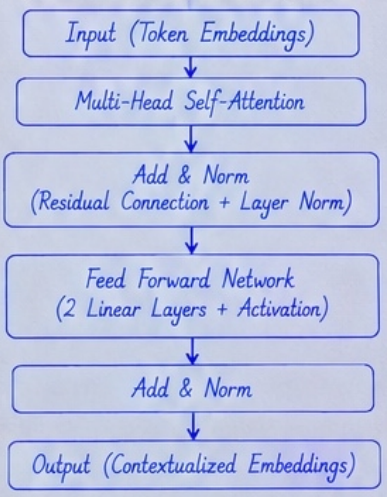
4.4 Encoder vs Decoder

Feature	Encoder	Decoder
Purpose	Understand input text	Generate output text
Input	Original text	Shifted target text
Attention	Self-attention	Masked self-attention + Cross-attention
Masking	Not required	Uses causal mask
Output	Contextual embeddings	Next token probabilities
Used in	BERT (encoder-only)	GPT (decoder-only), T5 (encoder-decoder)

4.5 Encoder-Only vs Decoder-Only vs Encoder-Decoder Models

Type	Architecture	Examples	Use Cases
Encoder-Only	Only encoder layers	BERT, RoBERTa	Text understanding, classification, NER
Decoder-Only	Only decoder layers (causal)	GPT, LLaMA, Gemini (decoder)	Text generation, chatbots, code generation
Encoder-Decoder	Both encoder and decoder	T5, BART	Translation, summarization, seq2seq tasks

4.6 Transformer Layer (High-Level)



4.7 Why Transformer Works Well for LLMs ?

- Self-attention captures global context in a single layer.
- Can model long-range dependencies effectively.
- Parallel processing reduces training time.
- Scales well with large model sizes and datasets.
- Flexible architecture (can be encoder-only, decoder-only or encoder-decoder).
- Works well with modern training techniques like pretraining and fine-tuning.

Rahul Yadav

5. Positional Encoding

5.1 What is Positional Encoding?

- Transformers use self-attention which is invariant to the order of tokens. It treats the input as a set of tokens.
- Positional encoding is a technique to inject information about the position (index) of each token in the sequence.
- It is added to the token embeddings before feeding them to the Transformer layers.
- This helps the model understand the sequence order and capture the meaning of the sentence correctly.

5.2 Why is it Needed?

- Self-attention alone does not know the order of tokens, so without positional encoding, "[The, cat, chased, the, dog]" and "[The, dog, chased, the, cat]" would look the same.
- Word order changes the meaning:
  - The cat chased the dog (cat is the subject)
  - The dog chased the cat (dog is the subject)
- So, positional encoding is essential to provide position information to the model.

5.3 Main Types of Positional Encoding

1. Sinusoidal Positional Encoding (original paper).
2. Learned Absolute Positional Embeddings (trainable embeddings).
3. Relative Positional Encoding (encode distance).
4. Rotary Position Embedding (RoPE) (used in LLaMA, GPT-NeoX, etc).

5.4 Sinusoidal Positional Encoding

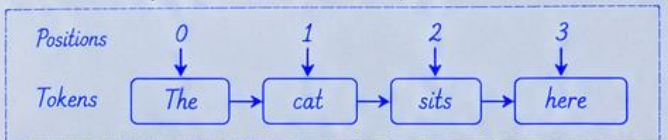
- Uses sine and cosine functions of different frequencies.
- It generates a fixed positional encoding vector for each position.
- Even positions use sine and odd positions use cosine.
- The embeddings have the same dimension as the token embeddings.

$$PE(pos, 2i) = \sin\left(\frac{pos}{10000^{2i/d_{model}}}\right)$$

$$PE(pos, 2i+1) = \cos\left(\frac{pos}{10000^{2i/d_{model}}}\right)$$

pos : token position (0, 1, 2, ...)
i : dimension index (0, 1, 2, ...)
d_{model} : embedding dimension (e.g., 512, 768, 1024)

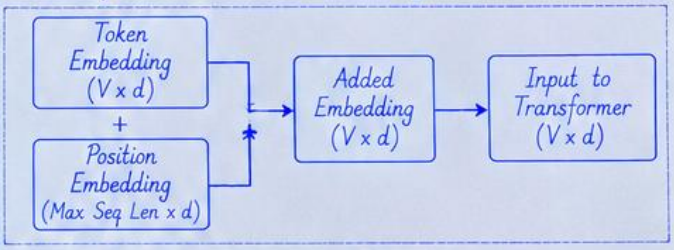
5.5 Simple Position Example



Each token embedding is added with its positional encoding vector: x = token_embedding + position_embedding

5.6 Learned Positional Embeddings

- Uses a trainable embedding matrix where each position has a learnable vector.
- The model learns the best position representations during training.
- It is added to the token embeddings just like sinusoidal encoding.
- **Advantage**: More flexible and can adapt to the task.
- **Limitation**: May not generalize well to longer sequences than seen in training.

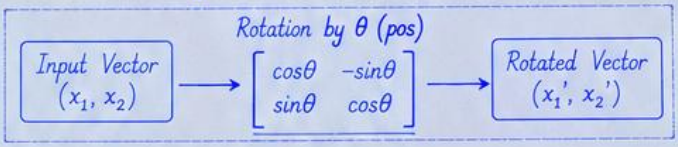


5.7 Relative Position Encoding

- Instead of using absolute positions, it encodes the relative distance between tokens.
- It helps the model understand how far apart tokens are, which is useful for many NLP tasks.
- Examples: Relative attention (Shaw et al.), Transformer-XL.
- Better for capturing long-range dependencies.

5.8 Rotary Position Embedding (RoPE)

- Rotates the query (Q) and key (K) vectors by a position-dependent angle in complex plane.
- It encodes relative position information implicitly through rotation.
- Helps the model capture long-range dependencies efficiently.
- Used in many modern decoder LLMs like LLaMA, GPT-NeoX.



5.9 Comparison of Methods

Method	Core idea	Strength	Limitation
Sinusoidal	Fixed sine & cosine functions	Simple, no trainable params, generalizes well	May not be optimal for specific tasks
Learned	Trainable position embeddings	Flexible, can adapt to task	May not generalize to longer sequences
Relative	Encode relative distance between tokens	Better for long-range dependencies	More complex implementation
RoPE	Rotate Q and K by position angle	Captures relative position, works well in modern LLMs	Slightly higher computational cost

5.10 Key Takeaways

- Transformers need positional information because self-attention is order invariant.
- Positional encoding is added to token embeddings.
- Different methods exist: sinusoidal, learned, relative and RoPE.
- Each method has its own advantages and trade-offs.
- RoPE and relative methods are widely used in modern LLMs.

Rahul Yadav

6. Attention Mechanism

6.1 What is Attention ?

- Attention allows the model to assign different weights to each input token based on its relevance to the current token.
- It helps the model focus on important parts of the input sequence instead of treating all tokens equally.
- Attention is content-dependent, which means the model decides the importance of each token dynamically.

Example : "The animal didn't cross the street because it was tired."

it → animal

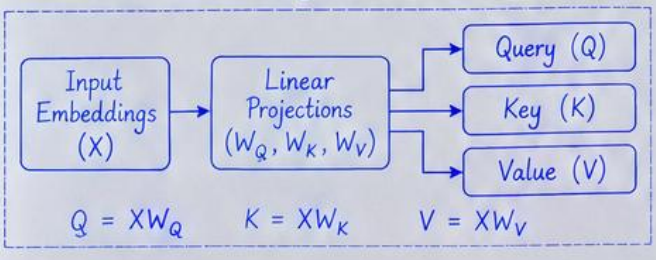
(Attention helps the model understand that "it" refers to "animal")

6.2 Why Attention is Needed ?

- Helps capture long-range relationships between tokens.
- Understands the contextual meaning of words.
- Allows dynamic focus on relevant tokens for each input.
- Enables parallel processing of all tokens, making it more efficient than RNNs.

6.3 Query, Key and Value (Q, K, V)

- **Query (Q)** : Represents what the current token is looking for (what information is needed).
- **Key (K)** : Represents what each token offers (used for matching with query).
- **Value (V)** : Contains the actual information to be combined (weighted and summed).



6.4 Attention Score

- The relevance between a query and a key is calculated using dot product.
- The score is scaled by $\sqrt{d_k}$ to prevent very large values, which can make training unstable.

$$score(Q, K) = \frac{QK^T}{\sqrt{d_k}}$$

d_k : dimension of key/query vectors

6.5 Softmax Weights

- The attention scores are converted into probabilities using softmax.
- This gives a normalized weight for each token.
- All attention weights for a given query sum to 1.

$$\alpha_{ij} = \frac{exp(s_{ij})}{\sum_k exp(s_{ik})}$$

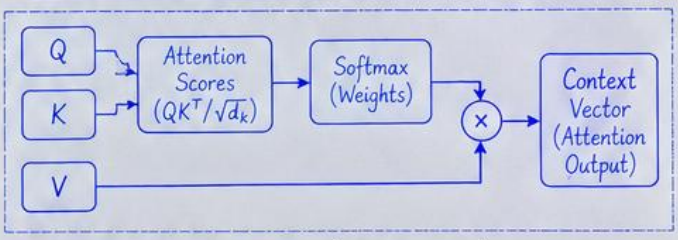
α_{ij} : attention weight from token i to token j
s_{ij} : attention score between token i and j

- Higher weight means the token is more relevant.
- Weights help the model decide which tokens to focus on.

6.6 Weighted Sum of Values

- The final output (context vector) is a weighted sum of value vectors.
- Each value vector is multiplied by its attention weight and then summed.

$$Attention(Q, K, V) = softmax\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$



6.7 Small Example (Illustrative)

Consider the sentence : "The cat slept"
Attention weights for the query token "cat" (illustrative values) :

Token (j)	Score (s_{ij})	Attention Weight (α_{ij})
The	0.2	0.20
cat	1.0	0.60
slept	0.1	0.20

The weights sum to 1 → 0.20 + 0.60 + 0.20 = 1.00
This means the token "cat" focuses more on itself, but also attends to "The" and "slept".

6.8 Key Takeaways

- Attention is content-dependent and assigns different importance to tokens.
- Q, K are used to compute relevance and V contains the information to be combined.
- Softmax converts scores into normalized weights.
- The final output is a weighted sum of value vectors.
- Attention can be restricted using masks (e.g., causal mask).
- Multi-Head Attention (multiple attention heads) is discussed in the next lesson.

Rahul Yadav

7. Self-Attention & Scaled Dot-Product

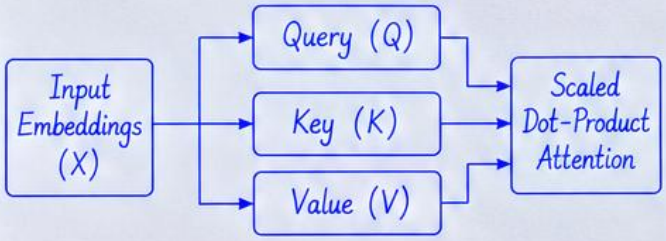
7.1 What is Self-Attention ?

- Self-Attention allows each token to attend to all other tokens in the same sequence.
- It helps the model understand the context and relationships between words, regardless of their distance in the sentence.
- It computes attention scores to find how much each token should focus on other tokens.
- It is the core mechanism used in Transformer models.

7.2 Intuition with Example

- Consider the sentence : "The cat slept on the mat".
- While processing the word "cat", the model may attend more to "The" and "slept" to understand its meaning.
- Self-Attention helps the model capture such relationships automatically.
- It is content-dependent, so attention weights are learned during training.

7.3 Architecture Overview



7.4 Query, Key and Value

- Query (Q) : What the current token is looking for.
- Key (K) : What each token offers.
- Value (V) : Actual information to be combined.
- These are linear projections of the input embeddings using learned weight matrices.

$$\underset{(n \times d_{model})}{Q} = \underset{(d_{model} \times d_k)}{XW_Q} \quad \underset{(d_{model} \times d_k)}{K = XW_K} \quad \underset{(d_{model} \times d_v)}{V = XW_V}$$

X : Input embeddings matrix (n = sequence length)
W_Q : Query weight matrix
W_K : Key weight matrix
W_V : Value weight matrix
d_k : Dimension of query/key vectors
d_v : Dimension of value vectors

7.5 Scaled Dot-Product Attention

- The attention score is computed using dot product between Query and Key vectors.
- The score is scaled by $\sqrt{d_k}$ to avoid very large values.
- Softmax is applied to get attention weights (probabilities).
- These weights are multiplied with Value vectors to get the final output (context vector).

$$\text{Attention}(Q, K, V) = \underbrace{\text{softmax}}_{\text{Dot Product (Similarity)}}\left(\underbrace{\frac{QK^T}{\sqrt{d_k}}}_{\text{Scaling (To prevent large values)}}\right)\underbrace{V}_{\text{Weighted Sum}}$$

Q : Query matrix $(n \times d_k)$
K : Key matrix $(n \times d_k)$
V : Value matrix $(n \times d_v)$
n : sequence length
d_k : dimension of query/key vectors

7.6 Step by Step Calculation (Small Example)

- Consider a simple example with 2 tokens and $d_k = 2$.
- We will compute attention scores, apply scaling, softmax and get the final output.

$$Q = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} \quad K = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} \quad V = \begin{bmatrix} 10 & 0 \\ 0 & 10 \end{bmatrix}$$

1. Compute QK^T :

$$QK^T = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}\begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$$

2. Scale by $\sqrt{d_k}$ ($d_k = 2$) :

$$\text{Scaled} = \frac{1}{\sqrt{2}}\begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} = \begin{bmatrix} 0.71 & 0 \\ 0 & 0.71 \end{bmatrix}$$

3. Apply softmax (row-wise) :

$$\text{Attention Weights} = \begin{bmatrix} 0.67 & 0.33 \\ 0.33 & 0.67 \end{bmatrix}$$

4. Multiply with V :

$$\text{Output} = \begin{bmatrix} 0.67 & 0.33 \\ 0.33 & 0.67 \end{bmatrix}\begin{bmatrix} 10 & 0 \\ 0 & 10 \end{bmatrix} = \begin{bmatrix} 6.7 & 3.3 \\ 3.3 & 6.7 \end{bmatrix}$$

7.8 Key Takeaways

- Self-Attention allows each token to attend to all tokens in the sequence.
- It uses Query, Key and Value projections.
- Attention scores are computed using scaled dot product and softmax.
- It helps capture long-range dependencies.
- It is the core mechanism behind Transformer models.

By Rahul Yadav

8. Multi-Head Attention

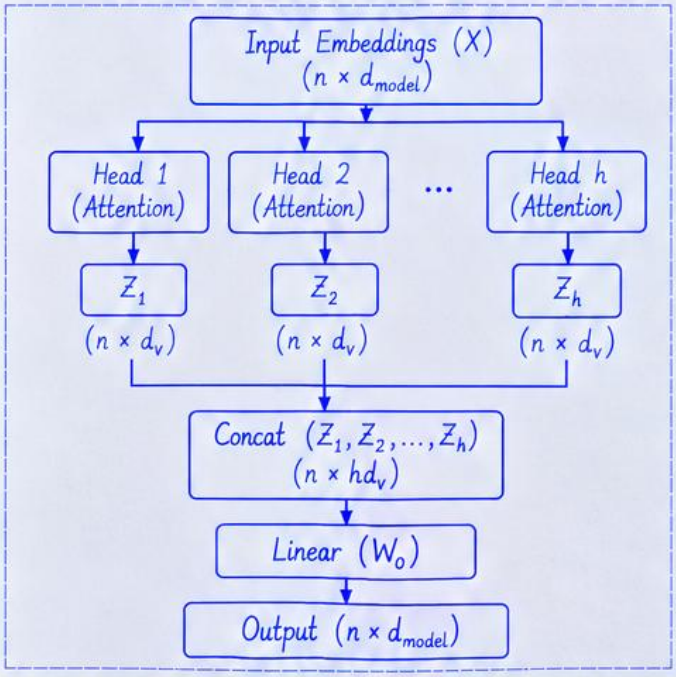
8.1 What is Multi-Head Attention ?

- Multi-Head Attention runs multiple attention heads in parallel.
- Each head learns different patterns and focuses on different types of relationships in the input sequence.
- Outputs from all heads are concatenated and passed through a final linear layer.
- It allows the model to capture richer representations compared to a single attention head.

8.2 Intuition with Example

- Consider the sentence : "The cat slept on the mat".
- One head may focus on subject-verb relationship ("cat" → "slept").
- Another head may focus on positional or syntactic information.
- Another head may focus on long-range dependency ("cat" → "mat").
- By using multiple heads, the model can capture all these relationships in parallel.

8.3 Architecture of Multi-Head Attention



8.4 Steps in Multi-Head Attention

1. Create multiple attention heads (h heads).
2. For each head, compute scaled dot-product attention with separate linear projections.
3. Concatenate the outputs of all heads.
4. Pass the concatenated output through a final linear layer.

8.5 Mathematical Formulation

- For each head i :

$$Q_i = XW_Q^i \qquad K_i = XW_K^i \qquad V_i = XW_V^i$$

$$Head_i = Attention(Q_i, K_i, V_i)$$

$$MultiHead(Q, K, V) = Concat(Head_1, \ldots, Head_h)\, W_o$$

$W_Q^i : (d_{model} \times d_k)$	d_{model} : model dimension
$W_K^i : (d_{model} \times d_k)$	d_k : dimension of query/key
$W_V^i : (d_{model} \times d_v)$	d_v : dimension of value
$W_o : (h \times d_v \times d_{model})$	h : number of heads
	X : input embeddings $(n \times d_{model})$
	n : sequence length

8.6 Scaled Dot-Product Attention (Per Head)

- For each head, attention is computed as :

$$Attention(Q_i, K_i, V_i) = softmax\left(\frac{Q_i K_i^T}{\sqrt{d_k}}\right) V_i$$

- Q_i, K_i, V_i are projections of input X using different weight matrices for each head.
- The scaling by $\sqrt{d_k}$ prevents very large values.

8.7 Concatenation and Final Linear Layer

- Outputs from all heads are concatenated :

$$Z = Concat(Head_1, Head_2, \ldots, Head_h)$$

- Z has dimension $(n \times hd_v)$.
- Then a final linear transformation is applied :

$$Output = Z\, W_o \qquad (n \times d_{model})$$

- This combines information from all heads.

8.8 Dimension Flow (Example)

- Let $d_{model} = 512$, $h = 8$ → $d_k = d_v = 64$
- Input : X $(n \times 512)$
- Each head output : $(n \times 64)$
- Concatenated output : $(n \times 8 \times 64) = (n \times 512)$
- After final linear layer : $(n \times 512)$

8.9 Key Takeaways

- Multi-head attention uses multiple attention heads in parallel.
- Each head learns different patterns and relationships.
- Outputs are concatenated and passed through a final linear layer.
- It helps the model capture richer representations and is a key component of Transformer architecture.

By Rahul Yadav

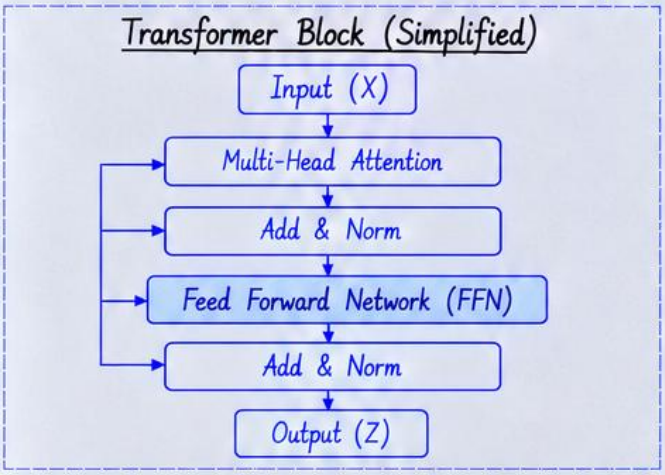
9. Feed Forward Networks

9.1 What is Feed Forward Network ?

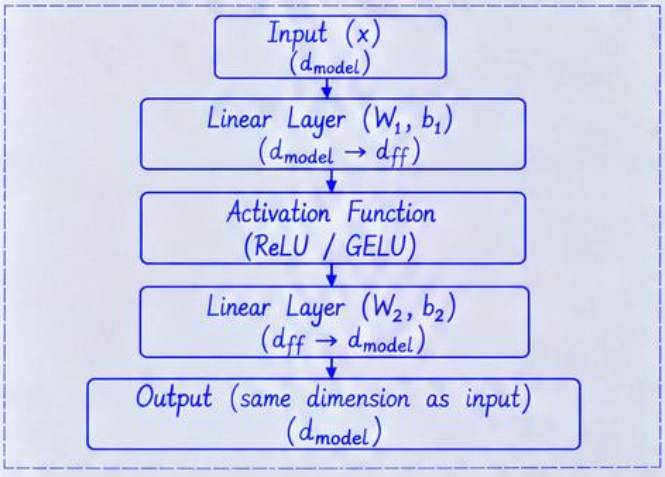
- Feed Forward Network (FFN) is a simple neural network where information flows in one direction (input → output).
- It consists of linear layers and non-linear activation functions.
- In Transformers, FFN is applied to each token position independently and identically.
- It helps the model learn complex, non-linear representations.

9.2 Position of FFN in Transformer

- In a Transformer block, FFN comes after the Multi-Head Attention layer.
- It is applied to each token separately.
- It is the same for all tokens (shared weights).



9.3 Architecture of Feed Forward Network



9.4 Dimensions used in Practice

Model	d_{model}	d_{ff}	Ratio (d_{ff} / d_{model})
BERT (Base)	768	3072	4
GPT-2 (Small)	768	3072	4
GPT-3	12288	49152	4
LLaMA	4096	11008	≈ 2.7

9.5 Mathematical Formulation

- The FFN consists of two linear transformations with a non-linear activation in between.

$$FFN(x) = W_2\,\sigma(W_1\,x + b_1) + b_2$$

x : input vector (dimension d_{model})
W_1 : weight matrix ($d_{model} \times d_{ff}$)
b_1 : bias vector (d_{ff})
σ : activation function (e.g., ReLU or GELU)
W_2 : weight matrix ($d_{ff} \times d_{model}$)
b_2 : bias vector (d_{model})
d_{model} : model dimension (e.g., 512)
d_{ff} : feed forward dimension (e.g., 2048)

9.6 Activation Functions

- Common activation functions used in Transformers:
- ReLU : $\sigma(x) = \max(0, x)$
- GELU : $\sigma(x) = x \cdot \Phi(x)$
  ($\Phi(x)$ is the Gaussian cumulative distribution)
- GELU is commonly used in modern models like BERT, GPT, LLaMA.

9.7 Example Calculation (Small)

- Let $d_{model} = 4$, $d_{ff} = 8$
- Input vector (x) : $\begin{bmatrix} 1 & 2 & 3 & 4 \end{bmatrix}^T$ (4×1)
- Step 1 : Linear transformation
  $h = W_1\,x + b_1$ (8×1)
- Step 2 : Apply activation (ReLU)
  $a = \max(0, h)$ (8×1)
- Step 3 : Second linear transformation
  $out = W_2\,a + b_2$ (4×1)

9.8 Key Takeaways

- Feed Forward Network is a two-layer neural network with a non-linear activation.
- It is applied to each token independently.
- It helps the model learn complex patterns.
- Typically uses ReLU or GELU.
- The intermediate dimension d_{ff} is larger than d_{model} (usually 4×).
- It is an important component of Transformer blocks and improves the model's representation power.

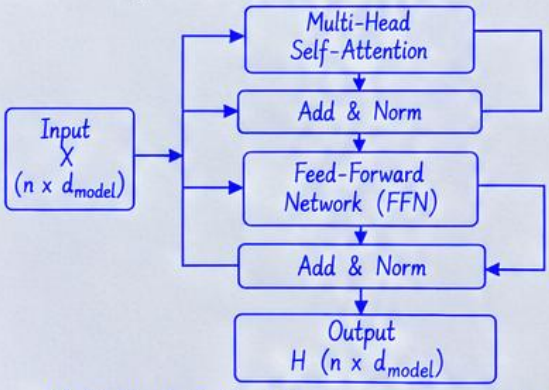
By Rahul Yadav

10. Transformer Blocks

10.1 What is Transformer Block ?

- A transformer block is a reusable layer that transforms token representations.
- It combines self-attention (to capture contextual information) with a feed-forward network (to process each token independently with non-linear transformations).
- It uses residual connections and layer normalization to stabilize training of deep networks.
- Multiple transformer blocks are stacked (N times) to build the full model (encoder or decoder).

10.2 Typical Encoder Block



- Multi-Head Self-Attention captures contextual information from all tokens.
- Add & Norm: Add residual connection and apply Layer Normalization.
- Feed-Forward Network (FFN) applies non-linear transformation to each token independently.
- The same structure is repeated for each encoder block.

10.3 Pre-LayerNorm vs Post-LayerNorm

Variant	Normalization placement	Practical note
Post-LN (Original)	After residual addition	Used in original Transformer (Vaswani et al.). Can be harder to train for very deep models.
Pre-LN	Before attention and FFN (inside sublayer)	Often more stable and easier to optimize for deep models. Common in modern implementations.

- Both variants are widely used and implementations may vary.

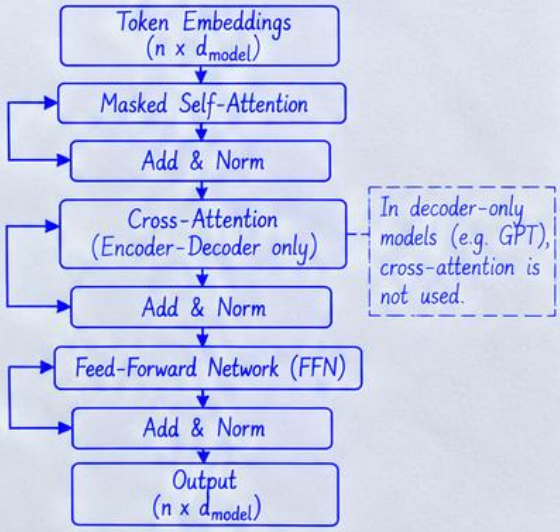
10.4 Residual Connection

- In Pre-LN variant, the sublayer output is added to the original input (residual connection).

$$Y = X + \text{Sublayer}(\text{LayerNorm}(X))$$

- The residual (identity) path helps gradient flow during training.
- It allows the model to preserve original information even after multiple transformations.

10.5 Decoder Block



10.6 Why Stack Multiple Blocks ?

- Each block refines the token representations by capturing more complex patterns.
- Lower layers may learn local or syntactic patterns while higher layers capture more abstract and semantic features (this is a general trend, not a strict rule).
- Increasing the number of blocks (depth) improves the model's capacity but also increases computational cost (more memory and time).

10.7 Tensor Dimensions Example

- Let sequence length = n, model dimension = d_{model}, feed-forward dimension = d_{ff}.

Component	Output Dimension
Input X (token embeddings)	(n x d_{model})
Multi-Head Attention output	(n x d_{model})
FFN intermediate (after first linear)	(n x d_{ff})
FFN output (after second linear)	(n x d_{model})
Residual addition	Requires same dimension (n x d_{model})

10.8 Common Interview Points

- Encoder block uses only self-attention, decoder block uses masked self-attention + cross-attention (optional).
- Attention mixes information across all tokens, while FFN processes each token position independently (with shared weights).
- Residual connections and layer normalization help training stability and enable deeper models.
- Masks control which tokens can be attended to (e.g. causal mask in decoder).

10.9 Key Takeaways

- A transformer block combines attention and FFN.
- Residual connections and layer normalization stabilize training.
- Multiple blocks are stacked to learn complex representations.
- Encoder and decoder blocks have different structures based on the task.

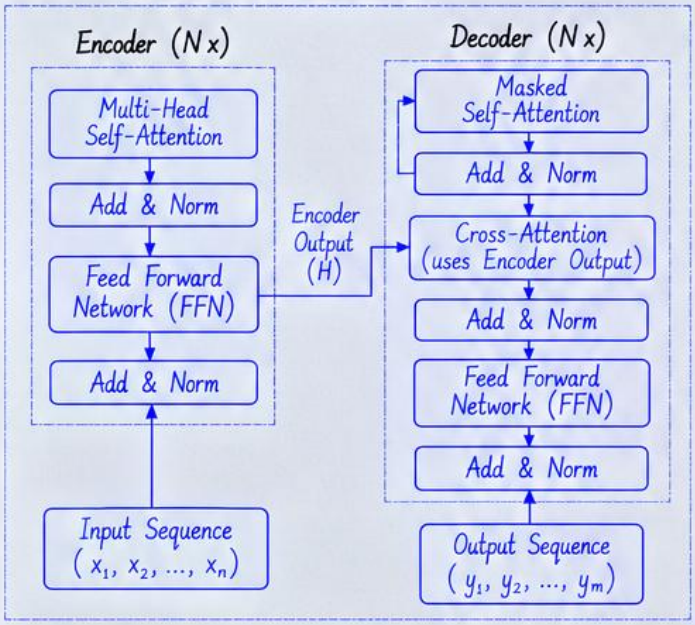
By Rahul Yadav

11. Encoder-Decoder Architecture

11.1 What is Encoder-Decoder Architecture ?

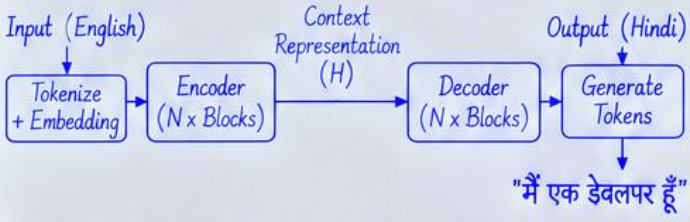
- Encoder-Decoder is a Transformer architecture used for sequence-to-sequence (seq2seq) tasks.
- It consists of two separate stacks:
  - Encoder : Understands the input sequence and converts it into a rich context representation.
  - Decoder : Generates the output sequence using the encoder representation and previously generated tokens.
- Commonly used in tasks like machine translation, text summarization, question answering, and text generation.

11.2 Overall Architecture



11.3 Data Flow (Example: Machine Translation)

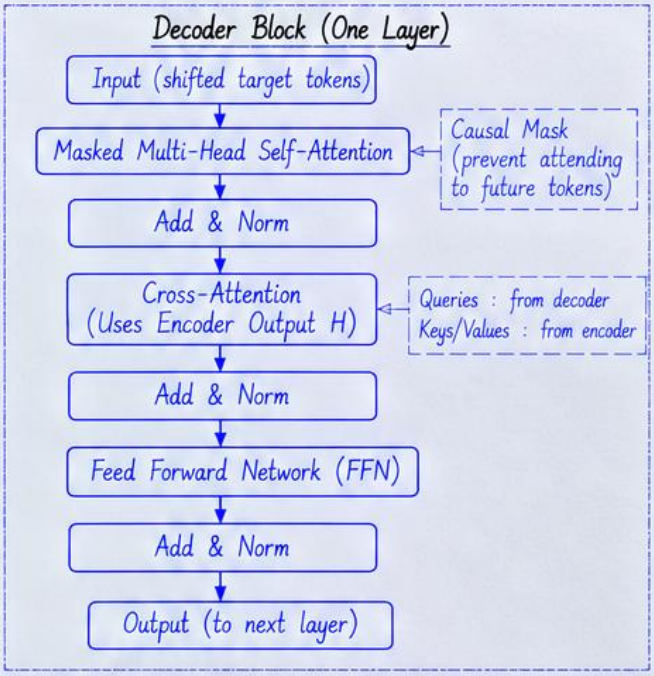
- Example : Translate "I am a developer" to Hindi.



11.4 Key Differences: Encoder vs Decoder

Aspect	Encoder	Decoder
Input	Source sequence	Target sequence (shifted right)
Self-Attention	Normal self-attention (looks at all tokens)	Masked self-attention (causal mask)
Cross-Attention	Not used	Uses encoder output as key and value
Purpose	Understand input	Generate output
Masking	Not required	Causal mask (prevent future tokens)
Common Tasks	Feature extraction	Sequence generation

11.5 Decoder Block in Detail



11.6 Cross-Attention Mechanism

- Cross-attention allows the decoder to attend to the encoder output at each generation step.
- It uses decoder hidden states as queries (Q) and encoder outputs as keys (K) and values (V).
- This helps the decoder focus on relevant parts of the input sequence while generating each token.

$$\text{CrossAttention}(Q, K, V) = \text{softmax}\left(\frac{Q K^T}{\sqrt{d_k}}\right) V$$

- Q : from decoder (target tokens)
- K, V : from encoder (source tokens)
- d_k : dimension of key vectors

11.7 Use Cases

- Machine Translation (English ↔ Hindi, etc.)
- Text Summarization
- Question Answering (seq2seq)
- Code Generation
- Dialog Systems
- Image Captioning (with vision encoder + text decoder)

11.8 Key Takeaways

- Encoder understands the input and creates a rich context representation.
- Decoder generates the output using masked self-attention, cross-attention, and FFN.
- Cross-attention connects encoder and decoder.
- Decoder uses causal masking to prevent looking at future tokens.
- Widely used for sequence-to-sequence tasks like translation and summarization.

By Rahul Yadav

12. Causal Masking

12.1 What is Causal Masking ?

- Causal masking (also called look-ahead masking) prevents the model from attending to future tokens in the sequence.
- It is used in decoder-based models (e.g., GPT).
- During training, the model can only see the current and previous tokens, not the next tokens.
- This ensures that the model generates text autoregressively (one token at a time).

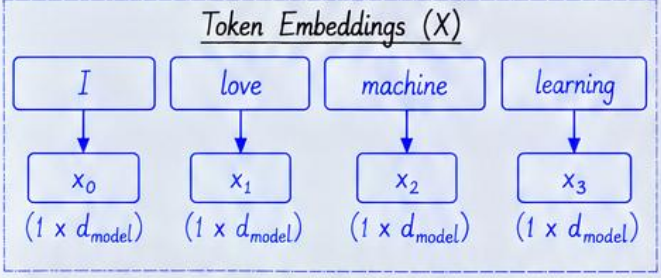
12.2 Why is it Needed ?

- In text generation, we should not allow the model to use future words to predict the current word.
- Without masking, the model can "cheat" by looking at future tokens, leading to unrealistic performance.
- Causal masking enforces the correct direction of information flow (left → right).

12.3 Example (Input Sequence)

Consider the input sequence : "I love machine learning"

Token indices : 0 1 2 3



12.4 Causal Mask Matrix

- We create a triangular mask matrix where positions can only attend to themselves and previous positions.
- Future positions are masked with $-\infty$ (very large negative value).
- After adding this mask to the attention scores, softmax will give 0 probability to future tokens.

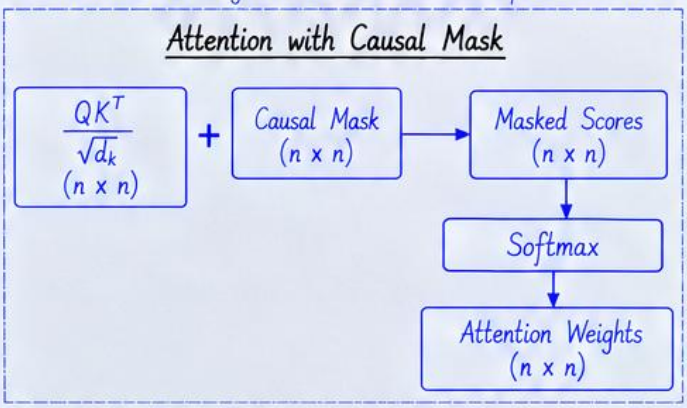
Causal Mask Matrix (4 x 4)

Position →	0	1	2	3
0	0	$-\infty$	$-\infty$	$-\infty$
1	0	0	$-\infty$	$-\infty$
2	0	0	0	$-\infty$
3	0	0	0	0

- 0 → token can attend (allowed)
- $-\infty$ → token cannot attend (masked)

12.5 How it Works in Attention ?

- In scaled dot-product attention, we add the causal mask to the attention scores before softmax.
- Masked positions get $-\infty$, so their probability becomes 0 after softmax.
- Each token only attends to current and previous tokens.



12.6 Mathematical Formulation

- Standard attention :

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

- With causal mask :

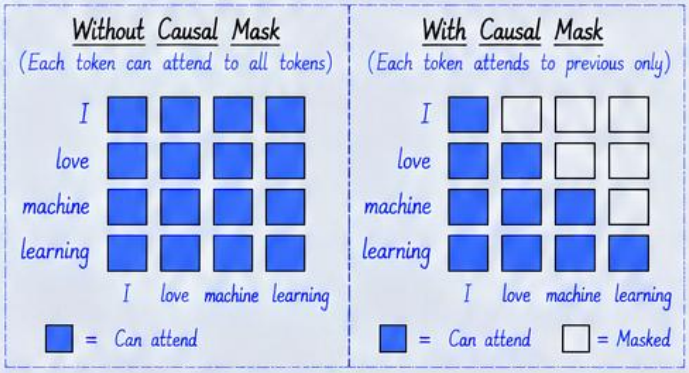
$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}} + M\right)V$$

- M is the causal mask matrix where :

$$M_{i,j} = \begin{cases} 0 & \text{if } j \le i \\ -\infty & \text{if } j > i \end{cases}$$

- If $j > i$, the token at position i cannot attend to token at position j.
- This ensures autoregressive generation.

12.7 Visualization



12.8 Key Takeaways

- Causal masking prevents the model from looking at future tokens.
- It is used in decoder-based models like GPT, LLaMA.
- Implemented by adding a triangular mask matrix with $-\infty$ to attention scores.
- Ensures autoregressive (left → right) text generation.

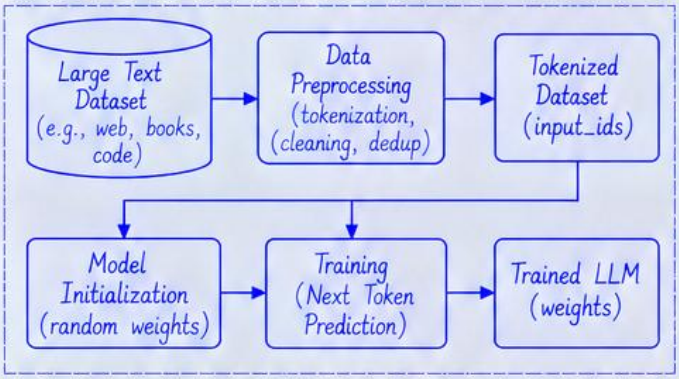
By Rahul Yadav

13. Training LLMs

13.1 Overview of LLM Training

- Training an LLM means teaching the model to predict the next token in a sequence.
- It learns patterns from large amounts of text data using self-supervised learning.
- The model is trained on a massive dataset (e.g., books, websites, code, etc.)
- The objective is to minimize the difference between the predicted token distribution and the actual next token.
- After training, the model can generate coherent, context-aware text.

13.2 Training Pipeline



13.3 Training Objective (Next Token Prediction)

- Given an input sequence of tokens $x_1, x_2, \ldots, x_n$, the model predicts the next token x_{n+1}.
- We model the probability distribution over the vocabulary using softmax.

Probability of next token

$$P(x_{n+1} \mid x_1, \ldots, x_n) = \text{softmax}\left(Wh_n + b\right)$$

h_n : hidden state from the last transformer layer
W : weight matrix (vocab_size × d_{model})
b : bias vector (vocab_size)
vocab_size : total number of tokens in vocabulary

13.4 Loss Function (Cross-Entropy Loss)

- We use Cross-Entropy Loss to measure the difference between predicted and actual token.
- For each position, we compute the loss:

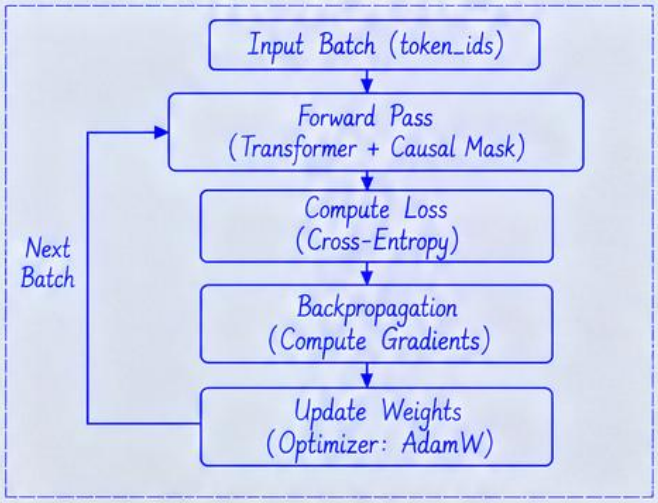
$$\mathcal{L} = -\frac{1}{N}\sum_{i=1}^{N} \log\left(P(x_i \mid x_1, \ldots, x_{i-1})\right)$$

N : total number of tokens in the training data
x_i : actual next token
$P(x_i \mid x_1, \ldots, x_{i-1})$: predicted probability of token x_i

13.5 Training Process (Step by Step)

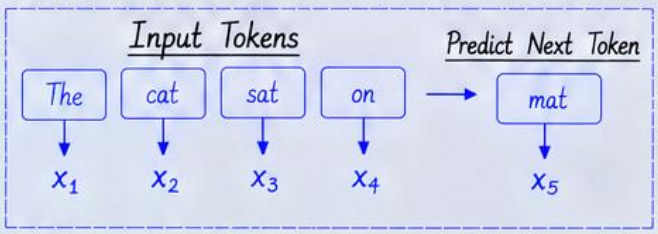
1. Input tokens are passed through the Transformer model (with causal masking).
2. The model outputs logits for the next token.
3. Softmax converts logits to probabilities.
4. Cross-entropy loss is computed with the actual token.
5. Gradients are calculated using backpropagation.
6. Model weights are updated using an optimizer (e.g., AdamW).
7. Repeat for many iterations (epochs) over the dataset.

13.6 Training Loop (Simplified)



13.7 Example: Next Token Prediction

- Input sequence : "The cat sat on"
- Tokenized : [The, cat, sat, on]
- The model predicts the next token → "mat"



13.8 Key Takeaways

- LLMs are trained using self-supervised learning with next token prediction.
- Causal masking ensures the model only looks at past tokens.
- Cross-entropy loss is used to train the model.
- Huge amounts of text data are used for training.
- Optimizers like AdamW and learning rate schedules help in stable training.
- The result is a model that can generate coherent, context-aware text for many tasks.

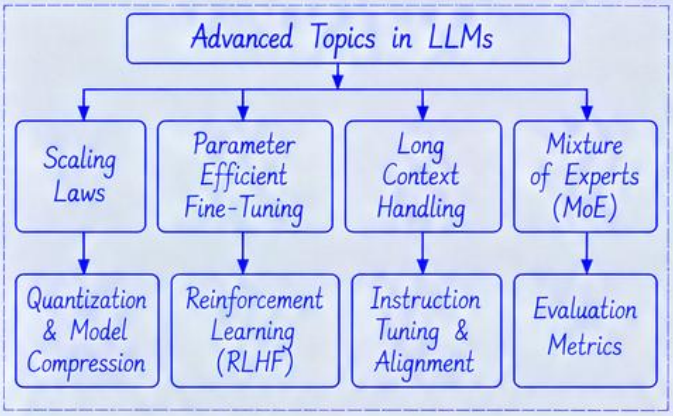
By Rahul Yadav

14. Advanced Topics in LLMs

14.1 What are Advanced Topics ?

- After understanding the basic Transformer architecture, there are several advanced concepts that improve the performance, efficiency, and capabilities of LLMs.
- These techniques help in training larger models, handling longer context, improving generalization, and enabling real-world applications.
- Advanced topics are essential to understand modern LLMs like GPT-4, LLaMA, Mistral, etc.

14.2 Overview of Key Advanced Topics



14.3 Scaling Laws

- Scaling laws show that model performance improves predictably with an increase in model size, dataset size, and compute.
- Larger models with more data and compute generally perform better, following a power-law relationship.

Scaling Law (Simplified)

$$L(N) = a \times N^{-\alpha} + b$$

- $L(N)$: model loss
- N : model size (parameters)
- a, b : constants
- α : scaling exponent (typically 0.05 – 0.1)

- As model size (N) increases, loss decreases.
- This shows why modern LLMs have billions of parameters.

14.4 Parameter Efficient Fine-Tuning (PEFT)

- Instead of fine-tuning all model parameters, PEFT methods train only a small number of additional parameters.
- This reduces compute and memory cost.
- Common PEFT techniques :

Technique	Key Idea	Advantage
LoRA	Add low-rank matrices to attention layers	Very efficient, widely used
Adapters	Add small adapter layers	Modular, parameter efficient
Prefix Tuning	Learn prefix tokens	Simple and effective
Prompt Tuning	Learn soft prompts	Works well for many tasks

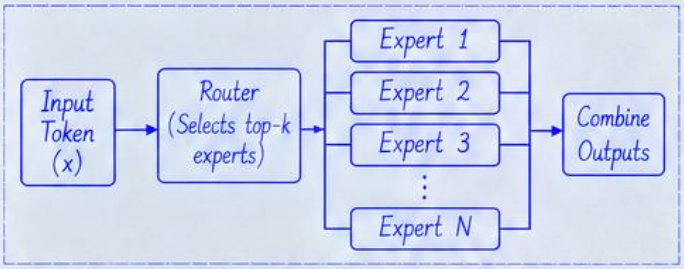
14.5 Long Context Handling

- Standard Transformers have fixed context length (e.g., 2K, 4K tokens).
- For long documents, advanced techniques are used to handle longer context.
- Methods :

Method	Key Idea	Example Models
Sparse Attention	Attend to only important tokens	Longformer, BigBird
Sliding Window Attention	Attend to local context	Mistral, Llama 2
Rotary Position Embedding (RoPE)	Better position representation	LLaMA, GPT-NeoX
Retrieval Augmented Generation (RAG)	Retrieve relevant chunks from DB	All modern LLM apps

14.6 Mixture of Experts (MoE)

- Instead of using all parameters for every input, MoE activates only a subset of expert networks.
- This increases model capacity while keeping compute cost low.



- Only a few experts (top-k) are activated for each token.
- Examples: Mixtral (8x7B), GLaM, Switch Transformer.

14.7 Quantization & Model Compression

- Reduces model size and memory usage by using lower precision (e.g., 8-bit, 4-bit) instead of 16/32-bit.
- Helps in faster inference and lower deployment cost.

Method	Description	Use Case
INT8 Quantization	Convert weights to 8-bit	Deployment
INT4 Quantization	Convert weights to 4-bit	Mobile / Edge
Pruning	Remove less important parameters	Reduce model size
Knowledge Distillation	Train smaller model to mimic larger model	Create lightweight models

14.8 Key Takeaways

- Advanced topics help in training larger, faster, and more efficient LLMs.
- PEFT, MoE, and quantization make LLMs practical for real-world use.
- Long context handling and alignment techniques improve model quality and safety.
- These concepts are widely used in modern LLMs like GPT-4, LLaMA, Mistral, Mixtral, etc.

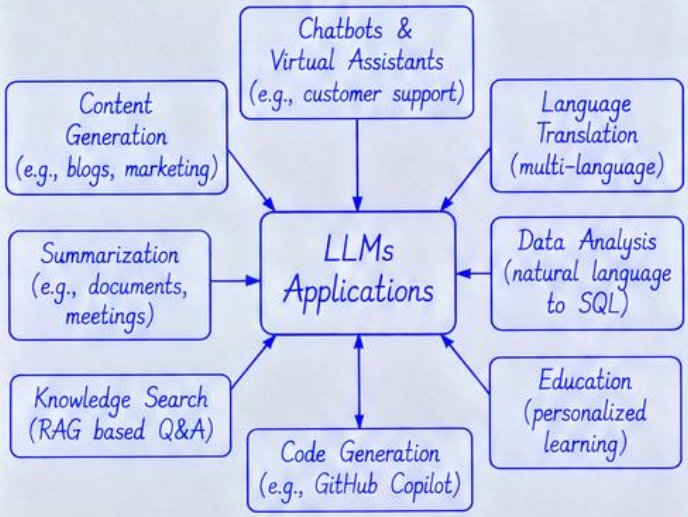
By Rahul Yadav

15. LLMs in Real World

15.1 Introduction

- LLMs are not just research models, they are widely used in real-world applications.
- They can understand, generate, summarize, classify and reason over human language.
- With proper system design, LLMs can be integrated into products, tools and business workflows to solve real problems.

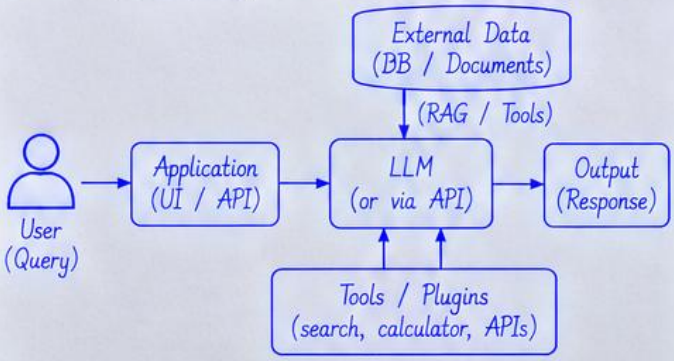
15.2 Real World Application Areas



15.3 Real World Use Case Examples

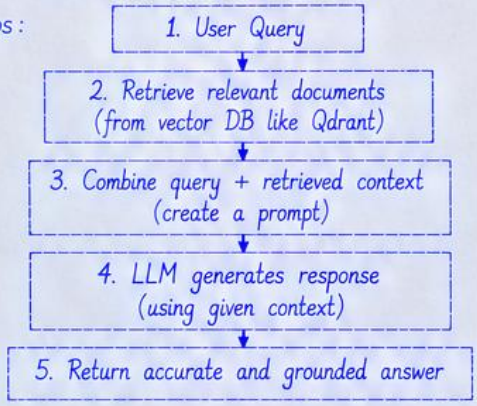
Domain	Use Case	Example
Banking	Customer support chatbot	Answer FAQs, account queries
Healthcare	Medical document summarization	Summarize patient reports
E-commerce	Product recommendation & description	Generate product content
Enterprise	Internal knowledge assistant	Search company documentation
Education	AI tutor	Explain concepts in simple language
Development	Code generation & review	Generate, debug and explain code

15.4 LLM Application Architecture



15.5 Retrieval Augmented Generation (RAG) (Real World Pattern)

- LLMs have knowledge only up to a certain time and may not know your private data.
- RAG combines LLM with external documents to provide accurate and up-to-date answers.
- Steps :



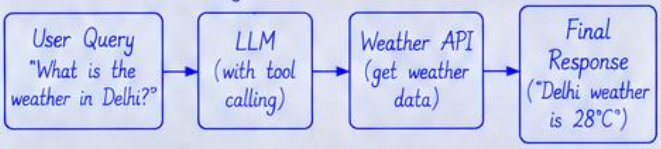
15.6 Prompt Engineering in Real World

- Good prompts help get accurate and useful responses.
- Different techniques are used based on the task.

Technique	Description
Zero-shot	Ask directly without examples.
Few-shot	Give a few examples in the prompt.
Role-based	Assign a role (e.g., You are a legal expert).
Chain-of-Thought	Ask the model to think step by step.
Structured Output	Ask for output in JSON / table format.

15.7 Working with Tools and APIs

- LLMs can call external tools to perform actions (e.g., search, calculator, database, APIs).
- This is usually done using Function Calling (or Tool Calling).



15.8 Challenges in Real World

- Hallucination (generating incorrect information).
- Keeping responses up-to-date.
- Handling sensitive or private data.
- Latency and cost for large models.
- Prompt injection and security risks.
- Ensuring factual, reliable and safe outputs.

15.9 Key Takeaways

- LLMs can be used in many real-world applications.
- RAG, prompt engineering and tool calling make them more powerful and reliable.
- They help automate tasks, improve productivity and create better user experiences.
- With proper design and safety measures, LLMs can be effectively used in production systems.

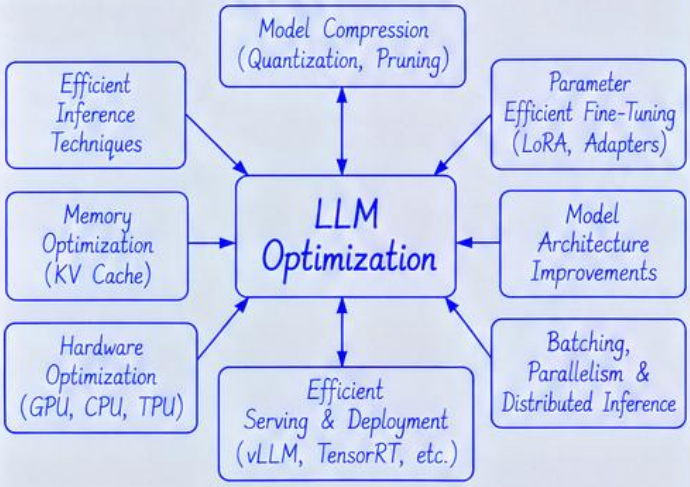
By Rahul Yadav

16. Performance & Optimization

16.1 Why Optimization is Important ?

- LLMs are very large and computationally expensive.
- Optimization helps to reduce memory usage, speed up inference, and lower deployment cost.
- It enables running LLMs on limited hardware (CPU/GPU) and in real-world applications (e.g., mobile, edge, production).
- Proper optimization improves latency, throughput and user experience without significantly degrading the quality of responses.

16.2 Areas of Optimization



16.3 Model Compression Techniques

Technique	Idea	Benefits	Use Case
Quantization	Convert 16/32-bit weights to 8/4-bit	Reduced memory & faster inference	Deployment on CPU / edge
Pruning	Remove less important weights	Smaller model size	Efficient deployment
Knowledge Distillation	Train a smaller model (student) using a larger model (teacher)	Faster inference with minimal quality loss	Run smaller models
Weight Sharing	Share weights across layers	Reduce memory usage	Large models

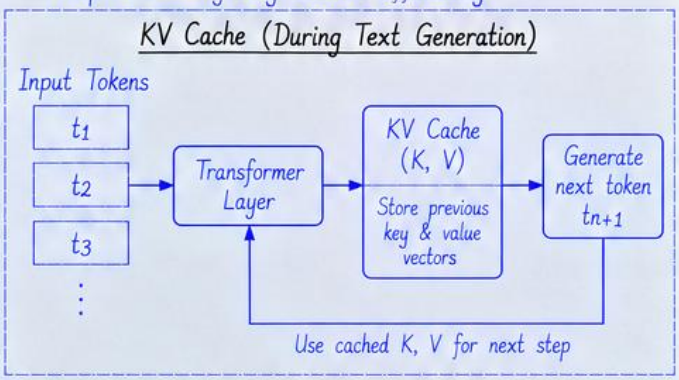
16.4 Quantization in LLMs

- Reduces the precision of model weights and activations (e.g., FP32 → INT8 or INT4).
- Significantly reduces memory usage and improves inference speed.
- Common quantization types :

Type	Description	Memory	Quality Impact
FP32	32-bit floating point	100%	Best (baseline)
FP16	16-bit floating point	~50%	Very small loss
INT8	8-bit integer	~25%	Small loss
INT4	4-bit integer	~12.5%	Moderate loss

16.5 KV Cache Optimization (Inference)

- In autoregressive generation, past key (K) and value (V) representations are cached to avoid recomputation.
- This reduces latency and increases token generation speed.
- Memory optimization techniques (e.g., paged attention) help in handling long context efficiently.



16.6 Efficient Inference Techniques

Technique	Description	Benefit
Batching	Process multiple requests together	Higher throughput
Paged Attention	Efficient memory management for long context	Lower memory usage
Speculative Decoding	Use a smaller model to generate draft tokens	Faster generation
Flash Attention	Optimized attention computation	Lower memory & faster inference
vLLM / TensorRT	High-performance inference engines	Better throughput and lower latency

16.7 Hardware Optimization

- Use GPUs/TPUs for faster matrix operations.
- Use optimized libraries like CUDA, cuDNN, TensorRT.
- Use multi-GPU or distributed inference for large models.
- Choose the right hardware based on model size and use case.

Hardware	Use Case	Advantage
CPU	Small models, low traffic	Cost effective
GPU (NVIDIA)	Training & inference	High parallelism
TPU (Google)	Large scale training	High performance
Edge Devices	On-device inference	Offline & low latency

16.8 Key Takeaways

- Optimization is essential to make LLMs practical, affordable and scalable.
- Techniques like quantization, pruning, distillation and efficient inference significantly reduce cost and latency.
- KV cache and advanced attention methods improve inference speed for long context.
- Using proper hardware and serving frameworks (vLLM, TensorRT, etc.) helps handle real-world traffic.
- The goal is to achieve the best trade-off between performance, cost and output quality.

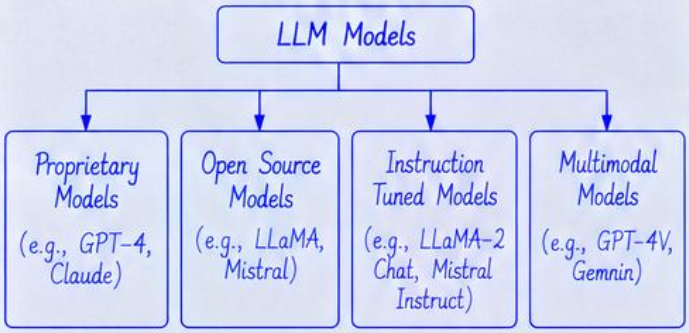
By Rahul Yadav

17. Popular LLM Models

17.1 Introduction

- There are many Large Language Models (LLMs) developed by different companies and research groups.
- Each model has different strengths, training data, architecture, and use cases.
- Choosing the right model depends on your task, cost, performance, and deployment needs.
- Here we cover some of the most popular LLM models used today in real-world applications.

17.2 Categories of LLM Models



17.3 Comparision of Popular LLM Models

Model	Developer	Type	Key Strength	Use Case
GPT-4	OpenAI	Proprietary	High reasoning & general capability	Chatbots, coding, analysis
GPT-3.5	OpenAI	Proprietary	Fast & cost-effective	General purpose tasks
Claude 3	Anthropic	Proprietary	Safer responses & long context	Enterprise applications
Gemini 1.5	Google	Proprietary	Multimodal & long context (1M tokens)	Text, image, video analysis
LLaMA 3	Meta	Open Source	High performance (open weights)	Research, customization
Mistral	Mistral AI	Open Source	Small size & high efficiency	Edge, local deployment
Mixtral	Mistral AI	Open Source (MoE)	High performance with MoE	Complex tasks with lower cost
Phi-3	Microsoft	Open Source	Lightweight & efficient	Edge devices, mobile apps
Cohere Command R	Cohere	Proprietary	RAG & enterprise focused	Business apps, RAG systems

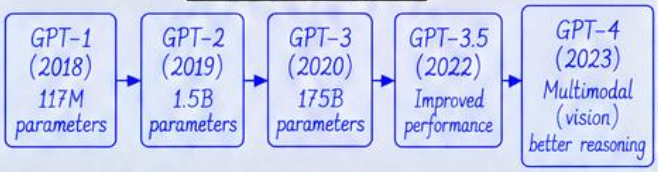
17.4 Open Source vs Proprietary Models

Open Source Models	Proprietary Models
• Model weights are publicly available. • Can be run locally and customized. • More control over data and privacy. • Examples: LLaMA, Mistral, Mixtral, Phi. • Good for research and custom applications.	• Model weights are not publicly available. • Access via API (paid or free tier). • Managed by the company (high reliability and support). • Examples: GPT-4, Claude, Gemini, Cohere. • Good for production use with minimal setup.

17.5 GPT Series (OpenAI)

- GPT (Generative Pre-trained Transformer) is a family of large language models developed by OpenAI.
- Trained on a massive amount of internet data.
- Known for strong reasoning, coding, and general knowledge capabilities.
- Used in ChatGPT, APIs, and many AI applications.

GPT Model Evolution



17.6 LLaMA (Meta)

- LLaMA (Large Language Model Meta AI) is an open source model developed by Meta.
- Designed for research and commercial use.
- Available in different sizes (e.g., 7B, 13B, 70B).
- LLaMA 2 and LLaMA 3 provide better performance and are widely used for fine-tuning.

Model	Parameters	Key Features	Use Case
LLaMA 1	7B, 13B, 33B	Base model	Research
LLaMA 2	7B, 13B, 70B	Better training data, chat models	Research, commercial
LLaMA 3	8B, 70B	Improved performance, multilingual	Production, fine-tuning

17.7 Mistral & Mixtral

- Mistral AI provides high performance open source models with efficient architecture.
- Diistral models are smaller in size but very powerful.
- Mixtral uses Mixture of Experts (MoE) architecture, which activates only a subset of parameters for each input.

Model	Parameters	Architecture	Key Points
Mistral 7B	7B	Decoder-only	High performance, lightweight
Mistral Instruct	7B	Instruction tuned	Good for chat & advanced tasks
Mixtral 8x7B	46.7B (8 experts)	Mixture of Experts	High performance with lower compute cost

17.8 Other Notable Models

Model	Developer	Key Features	Use Case
Claude 3	Anthropic	Long context, safer responses	Enterprise, analysis
Gemini 1.5	Google	Multimodal, 1M tokens	Text, image, video
Phi-3	Microsoft	Small, efficient	Edge, local deployment
Command R	Cohere	RAG optimized	Enterprise applications

17.9 Key Takeaways

- Different LLM models are designed for different use cases.
- Proprietary models (GPT, Claude, Gemini) provide high performance with easy API access.
- Open source models (LLaMA, Mistral, Mixtral, Phi) give more control and customization.
- Choose the right model based on your requirements like performance, cost, data privacy and deployment needs.

By Rahul Yadav

18. Challenges & Limitations

18.1 Introduction

- LLMs are very powerful, but they also have several challenges and limitations.
- It is important to understand these to use them effectively in real-world applications.
- These limitations exist due to data, model design, training methods, and the nature of language modeling (predicting next token).

18.2 Major Challenges Overview



18.3 Hallucination (Generating Incorrect Information)

- LLMs sometimes generate factually incorrect or made-up information, even if it sounds confident.
- This happens because models predict the next token based on patterns, not actual understanding.

Example	
User:	"Who won the FIFA World Cup in 2026?"
LLM:	"The 2026 FIFA World Cup was won by Brazil." (This may be incorrect if the event has not happened)

18.4 Bias and Fairness

- LLMs can inherit biases present in the training data.
- This can lead to unfair, toxic, or discriminatory outputs across gender, race, religion, or other attributes.

Example	
User:	"Is software engineering a good career for girls?"
Biased Output:	"It is generally better suited for boys because they are more logical and technical."

(This is a biased and unfair response)

18.5 Limited Real-time Knowledge

- Most LLMs have a knowledge cutoff date (e.g., GPT-4: April 2023).
- They do not have access to real-time internet data unless integrated with tools (e.g., search, APIs).
- So, they may not know the latest news, events, or updates.

18.6 Context Length Limitations

- LLMs can only process a limited number of tokens at a time (context window).
- For example:

Model	Typical Context Length
GPT-3.5	4K – 16K tokens
GPT-4	8K – 128K tokens
Claude 3	200K tokens
LLaMA 3	8K – 70B (varies)

- If the input is longer than the context window, older information may be lost.
- This can affect long documents, conversations, and complex tasks.

18.7 High Computational Cost

- Training and running LLMs require large computational resources (GPUs/TPUs).
- Inference for large models can be expensive in terms of cost, latency, and energy consumption.
- This makes it challenging for small organizations or personal use at large scale.

18.8 Lack of True Reasoning and Common Sense

- LLMs are good at pattern recognition but do not truly understand the world like humans.
- They can struggle with complex reasoning, math, and tasks requiring deep domain knowledge.

Example (Logical Reasoning)	
User:	"If all birds can fly and a penguin is a bird, can a penguin fly?"
LLM:	"Yes, a penguin can fly because all birds can fly."

(Incorrect – shows lack of common sense)

18.9 Prompt Sensitivity

- Small changes in the prompt can lead to very different outputs.
- It can be difficult to get consistent and reliable responses.

Example	
Prompt 1:	"Summarize climate change."
Output:	(Good and detailed summary)
Prompt 2:	"Summarize climate change in one line."
Output:	(Very short and different focus)

18.10 Key Takeaways

- LLMs are powerful but have limitations.
- Be aware of issues like hallucination, bias, limited knowledge, and high cost.
- Use techniques like RAG, human review, prompt engineering, and evaluation to mitigate these challenges.
- Choose the right model and tools based on your use case. and requirements.

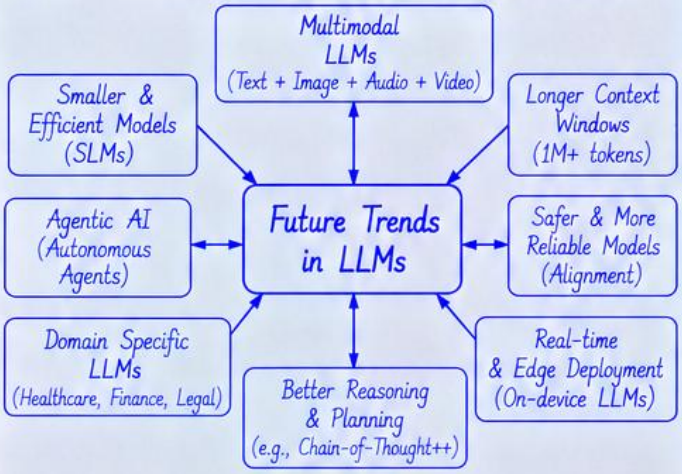
By Rahul Yadav

19. Future Trends & Career in LLMs

19.1 Introduction

- LLMs are evolving very fast and will continue to improve in the coming years.
- New techniques, architectures, and applications are being developed to make them more capable, safer, and efficient.
- Understanding future trends helps us prepare for opportunities and challenges in the AI industry.

19.2 Future Trends in LLMs (Overview)



19.3 Multimodal LLMs

- Future models will understand and generate multiple types of data like text, images, audio, and video.
- Examples: GPT-4V, Gemini, Claude 3, LLaVA, Qwen-VL.
- Use cases: image understanding, video analysis, speech-to-text, and more natural human-computer interaction.

Model	Modalities	Use Case
GPT-4V	Text + Image	Image understanding, visual Q&A
Gemini 1.5	Text + Image + Video	Multi-modal reasoning
Claude 3	Text + Image	Document and image analysis
LLaVA	Text + Image	Open source vision model

19.4 Smaller & Efficient Models (SLMs)

- Not all use cases require very large models.
- Smaller models can be fine-tuned for specific tasks and run efficiently on limited hardware.
- Examples: Phi-3, Gemma, Mistral 7B, Llama 3 8B.

Model	Parameters	Advantage	Use Case
Phi-3	3.8B	Small & efficient	Edge / Local use
Gemma	2B / 7B	Lightweight	Research / Custom apps
Mistral 7B	7B	High performance	Commercial use
Llama 3 8B	8B	Open source	Fine-tuning / Production

19.5 Agentic AI (LLM Agents)

- LLM agents can plan, use tools, and take actions autonomously to complete complex tasks.
- They can interact with APIs, databases, and external tools.
- Examples: AutoGPT, LangChain agents, OpenAI Assistants.
- Use cases: research, automation, data analysis, personal assistants.

19.6 Domain Specific LLMs

- LLMs are being specialized for specific industries like healthcare, finance, legal, education, and automotive.
- These models perform better because they are trained on domain-specific data.
- Examples: Med-PaLM (healthcare), BloombergGPT (finance), Legal LLMs (law), BioGPT (biology).

Domain	Example Model	Use Case
Healthcare	Med-PaLM	Medical Q&A, research support
Finance	BloombergGPT	Financial analysis, reports
Legal	Legal LLMs	Contract analysis, legal research
Education	Khanmigo	Personalized learning
Automotive	DriveLM	Autonomous driving support

19.7 Longer Context Windows

- Future models will support much larger context lengths (e.g., 1M+ tokens).
- This enables processing of long documents, entire codebases, and multi-turn conversations without losing information.
- Examples: Gemini 1.5 (1M tokens), Claude 3 (200K tokens).

Model	Context Length	Benefit
GPT-4	128K	Long documents
Claude 3	200K	Better long context understanding
Gemini 1.5	1M	Very long documents, video, code

19.8 Safer & More Reliable Models

- Future LLMs will focus more on safety, alignment, factual accuracy, and reducing harmful outputs.
- Techniques: RLHF, Constitutional AI, red-teaming, better evaluation methods.
- Goal: make LLMs trustworthy and production-ready.

Technique	Purpose	Example
RLHF	Align with human values	Used in ChatGPT
Constitutional AI	Follow safety principles	Used in Claude
Red-teaming	Find and fix weaknesses	Internal testing
Factuality Evaluation	Reduce hallucination	RAG + better training

19.9 Real-time & Edge Deployment

- LLMs are moving towards real-time and on-device execution for privacy, low latency, and offline use.
- Examples: Llama.cpp, Phi-3, Gemma, Mistral (quantized models).
- Use cases: mobile apps, edge devices, IoT, automotive.

Approach	Advantage	Use Case
Quantized Models	Smaller size	Run on mobile / edge
On-device LLMs	No internet needed	Privacy sensitive apps
Edge Inference	Low latency	Real-time applications

19.10 Key Takeaways

- LLMs will become more multimodal, efficient, safe, and capable in the future.
- Domain-specific models and agentic AI will unlock new possibilities.
- Longer context windows and edge deployment will make LLMs more practical.
- There are huge career opportunities in research, development, and application of LLMs.

By Rahul Yadav